

Module 4: Supervised Learning
Submodule: 4.8 Kernels
Lecture: Computational Efficiency of Kernels

XCS229 Machine Learning

[Tengu Ma]

Contents

I	Introduction	1
II	Representing θ	1
	Induction Proof for the Representation of θ	2
III	Kernel Function	5
IV	Summary	6

I. Introduction

In this lecture, we explore the **kernel method** to speed up the algorithm under consideration.

II. Representing θ

For simplicity, at the beginning time step (i.e., $t = 0$ iteration) we initialize:

$$\theta^{(0)} = 0$$

We start by asserting that the vector θ can always be expressed as a linear combination of features:

$$\theta = \sum_{i=1}^n \beta_i \phi(x^{(i)})$$

for some $\beta_1, \dots, \beta_n \in \mathbb{R}$. (No condition is needed if $\theta^0 = 0$.) This representation implies that we need only store the scalars β_i instead of the entire vector θ . β is now our “surrogate” for θ .

Let p be the dimensionality of θ which is significantly larger than n (i.e., $p \gg n$). This reduction from p to n parameters helps minimize storage and computational overhead.

Induction Proof for the Representation of θ

We will demonstrate how the vector θ can be represented as a linear combination of feature vectors using an induction proof.

- Base Case at Iteration 0: At iteration 0, we initialize:

$$\theta^{(0)} = 0 = \sum_{i=1}^n 0 \cdot \phi(x^{(i)})$$

Here, all coefficients $\beta_i^{(0)}$ are zero, which affirms that θ is indeed a linear combination of the feature vectors, as required.

- Step for Iteration 1: At iteration 1, we update θ according to:

$$\begin{aligned} \theta^{(1)} &:= \theta^{(0)} + \alpha \sum_{i=1}^n \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right) \phi(x^{(i)}) \\ &= 0 + \alpha \sum_{i=1}^n y^{(i)} \phi(x^{(i)}) \\ &= \sum_{i=1}^n \underbrace{\alpha y^{(i)}}_{\beta_i \text{ at iteration 1}} \phi(x^{(i)}) \end{aligned}$$

where θ^T in the first row is at $t = 0$. This establishes that $\theta^{(1)}$ is a linear combination of the feature vectors.

- Induction Hypothesis: Assume at iteration t , the representation holds:

$$\theta^{(t)} = \sum_{i=1}^n \beta_i \phi(x^{(i)})$$

- Induction Step at Iteration $t + 1$: Show this holds for iteration $t + 1$:

$$\begin{aligned} \theta^{(t+1)} &:= \theta^{(t)} + \alpha \sum_{i=1}^n \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right) \phi(x^{(i)}) \\ &= \sum_{i=1}^n \beta_i \phi(x^{(i)}) + \alpha \sum_{i=1}^n \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right) \phi(x^{(i)}) \\ &= \sum_{i=1}^n \underbrace{(\beta_i + \alpha (y^{(i)} - \theta^T \phi(x^{(i)})))}_{\text{new } \beta_i} \phi(x^{(i)}) \end{aligned}$$

where θ^T in the first row is at iteration t . This equation indicates that at iteration $t + 1$, θ remains a linear combination of the feature vectors with updated coefficients β_i .

By induction, we showed that at each iteration, θ can be represented as:

$$\theta = \sum_{i=1}^n \beta_i \phi(x^{(i)})$$

This representation holds true for all iterations, demonstrating that the algorithm consistently maintains the linear combination format as it updates through iterations. As such, we only need to store n parameters as opposed to n parameters.

In order to implement this, we derive the update rule of the coefficients $\beta_1, \dots, \beta_n$. Using the equation above, we see that the new β_i depends on the old one via:

$$\beta_i := \beta_i + \alpha \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right)$$

Here we still have the old θ on the RHS of the equation. Replacing θ using $\theta = \sum_{j=1}^n \beta_j \phi(x^{(j)})$ gives:

$$\forall i \in \{1, \dots, n\}, \beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^n \beta_j \phi(x^{(j)})^T \phi(x^{(i)}) \right)$$

We often rewrite

$$\phi(x^{(j)})^T \phi(x^{(i)})$$

as

$$\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$$

to emphasize that it's the inner product of the two feature vectors. Thus

$$\forall i \in \{1, \dots, n\}, \beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^n \beta_j \langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle \right)$$

Significance: This allows us to update β_i without direct dependence on θ . Viewing β_i as the new representation of θ , we have successfully translated the batch gradient descent algorithm into an algorithm that updates the value of β iteratively.

It may now appear that at every iteration, we still need to compute the values of $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ for all pairs of i, j , each of which may take roughly $O(p)$ operations. However, two important properties come to rescue:

- **Pre-computation:** We can pre-compute the pairwise inner products $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ for all pairs of i, j before the loop starts, meaning that we only need to do this once at the very beginning.

- **Efficient Feature Map Calculation:** For the feature map ϕ , computing $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ can often (but not always) be efficient and does not necessarily require computing $\phi(x^{(i)})$ explicitly.

Using the ϕ from the previous lecture and choosing x and z as arbitrary indices, we begin with:

$$\langle \phi(x), \phi(z) \rangle = \begin{bmatrix} 1 & x_1 & x_2 & \cdots & x_1^2 & x_1x_2 & \cdots & x_2x_1 & \cdots & x_1^3 & \cdots \end{bmatrix} \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_1^2 \\ x_1x_2 \\ x_1x_3 \\ \vdots \\ x_2x_1 \\ \vdots \\ x_1^3 \\ x_1^2x_2 \\ \vdots \end{bmatrix}$$

This is because:

$$\begin{aligned} \langle \phi(x), \phi(z) \rangle &= 1 + \sum_{i=1}^d x_i z_i + \sum_{i,j \in \{1, \dots, d\}} x_i x_j z_i z_j + \sum_{i,j,k \in \{1, \dots, d\}} x_i x_j x_k z_i z_j z_k \\ &= 1 + \sum_{i=1}^d x_i z_i + \left(\sum_{i=1}^d x_i z_i \right) \left(\sum_{i=1}^d x_i z_i \right) \\ &\quad + \left(\sum_{i=1}^d x_i z_i \right) \left(\sum_{i=1}^d x_i z_i \right) \left(\sum_{i=1}^d x_i z_i \right). \end{aligned}$$

Therefore, we can express this as:

$$\langle \phi(x), \phi(z) \rangle = 1 + \sum_{i=1}^d x_i z_i + \left(\sum_{i=1}^d x_i z_i \right)^2 + \left(\sum_{i=1}^d x_i z_i \right)^3$$

Thus, to compute $\langle \phi(x), \phi(z) \rangle$, we can first compute $\langle x, z \rangle$ with $O(d)$ time and then take a constant number of operations to compute

$$\langle \phi(x), \phi(z) \rangle = 1 + \langle x, z \rangle + \langle x, z \rangle^2 + \langle x, z \rangle^3 \quad (1)$$

You might wonder if you really saved any time. Not yet because if you want to compute a new β from the old β , you still have to conduct this

inner product. This inner product between two p -dimensional vectors still takes $O(p)$ time. You still have the sum over n indices, so it's still $O(n \cdot p)$. Therefore, you haven't really saved much since ϕ still shows up here.

Here is where the magic happens. The inner product can be preprocessed for all pairs of i, j . This means you can compute it once and not have to recompute it every time since this quantity does not really change over time in your algorithm. By "over time," we mean as your algorithm is running this quantity doesn't change at all.

Additionally, the other important thing is that oftentimes you can compute this inner product faster than you thought. The naive implementation would be to just compute this factor and that factor and take the inner product, but in many cases you can do some math to speed it up.

This can be computed without even evaluating ϕ explicitly. If you have to evaluate each of these factors, you already wasted $O(p)$ time. However, sometimes you can do some math to not even evaluate ϕ .

The reason you can save on degrees of freedom, despite having p degrees of freedom in your θ , is because when your dataset is not as large as p , you cannot utilize all of them. You are not fundamentally using all the dimensionality or all the degrees of freedom in θ . If you have n samples, the maximum degrees of freedom you can achieve is n samples, and that's why you can save without any approximation.

This approach is really just a reimplementaion of the same algorithm. We didn't lose any information; it's not like you're doing any approximation at all.

Now, let's say this clearly: there is a notation that people use to call this a so-called **kernel function**. The kernel function allows us to compute the inner product in a high-dimensional space without explicitly mapping data points into that space, making many algorithms more efficient and scalable.

III. Kernel Function

We define the **Kernel** corresponding to the feature map ϕ as a function that maps $\mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ satisfying:

$$K(x, z) \triangleq \langle \phi(x), \phi(z) \rangle$$

$\mathcal{X}$ is the space of the input x . In our running example, $\mathcal{X} = \mathbb{R}^d$.

The final kernel algorithm is as follows:

Algorithm 1 Kernel Algorithm

1. Compute all the values $K(x^{(i)}, x^{(j)}) \triangleq \langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$ using equation (1) for all $i, j \in \{1, \dots, n\}$. Set $\beta := 0$ with $\beta \in \mathbb{R}^n$.
2. **Loop:**

$$\forall i \in \{1, \dots, n\}, \beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^n \beta_j K(x^{(i)}, x^{(j)}) \right)$$

In vector notation, letting K be an $n \times n$ matrix with $K_{ij} = K(x^{(i)}, x^{(j)})$, we have

$$\beta := \beta + \alpha(\vec{y} - K\beta)$$

Now, let's analyze the runtime for this particular polynomial feature.

- For computing K , we are using a formula that requires $O(d)$ time for each pair. Therefore, for all pairs (i, j) , the total is $O(n^2d)$.
- The loop updates each β_i , thus incurs $O(n^2)$ per iteration.

It's significant that there's no p involved anymore; it's just n and d . If n is small enough, particularly when $n^2 < p$, this is advantageous. Previously, each iteration took $n \times p$, but now it takes n^2 . This method becomes especially useful when p is large compared to n , as in cases where p could be exponentially larger, such as d^3 .

In conclusion, while the total runtime reflects $O(n^2d) + O(n^2T)$ where T is the number of iterations, the challenge remains in estimating T . It can vary significantly based on the specific problem. To make a meaningful comparison, one must generally consider that the absence of p implies a potential reduction in complexity and thus a more efficient algorithm, especially when p is large.

IV. Summary

In this lecture, we have discussed several key points regarding kernels and their computational efficiency:

- Kernels provide a method to compute inner products in high-dimensional spaces without explicitly mapping data points into that space.
- We have derived the representation of θ as a linear combination of feature vectors, emphasizing the importance of the coefficients β .

- The use of kernels allows for significant computational savings, particularly when transitioning from a high-dimensional space p to a lower-dimensional representation n .
- Pre-computation of inner products between feature vectors enhances the efficiency of algorithms using kernels.
- Our final kernel algorithm demonstrates how to update coefficients iteratively, resulting in a more efficient implementation of machine learning algorithms.